
Multi-Agent AutoML

Research & Build Plan

*Schema-Driven Structured Outputs for
Reliable LLM-Based AutoML Pipelines*

Institution: École Nationale Supérieure d'Informatique (ESI), Algiers

Target Venue: AutoML-Conf 2026 — Late-Breaking Track

Submission Deadline: July 15, 2026

Build Timeline: 4–5 Weeks

This document is a self-contained technical guide and sprint plan.
Treat it as a living reference — update checkboxes as tasks complete.

Contents

1	Research Overview	2
1.1	The Core Research Claim	2
1.2	Why This is Novel	2
1.3	Scope Statement	2
1.4	Metrics That Matter for the Paper	3
2	System Architecture	3
2.1	Current Components (Working)	3
2.2	Target Architecture (After Build)	3
2.3	Key Design Decisions — Do Not Change These	3
3	Coding Practice — Follow This Order Every Time	4
3.1	File Structure (Target)	4
4	Sprint Plan — Week by Week	5
4.1	Week 1 — Refactor Foundation	5
4.2	Week 2 — New Agents	7
4.3	Week 3 — Experiments & Baseline	9
4.4	Week 4 — Explainability, Ablation Flags & Poster	10
4.5	Week 5 — Paper & Submission (Buffer)	12
5	LLM Provider Setup	12
5.1	Free Models to Use	12
6	Datasets Reference	12
7	Reading List	13
7.1	Must Cite (Related Work)	13
7.2	Background Reading	13
8	Timeline Summary	13

1. Research Overview

1.1. The Core Research Claim

Core Research Claim

Primary Research Question (Option A):

To what extent do schema-constrained LLM outputs reduce plan invalidity and execution failures in automated machine learning pipelines, compared to free-form prompting, on tabular classification benchmarks?

Secondary Finding (Option C, if Plan Validator catches enough bugs):

How does schema-driven inter-agent communication affect the correctness and convergence rate of multi-agent AutoML systems?

1.2. Why This is Novel

Traditional AutoML systems (Auto-sklearn, FLAML, TPOT) use schemas *internally* — the configuration space is hardcoded by the system designer, not communicated to/from a language model. Recent LLM-based AutoML papers (AIDE, AutoML-GPT, DS-Agent) use free-form prompting and measure final accuracy only — **none treat plan validity rate and execution success rate as first-class metrics.**

Your novelty is the combination of:

- Pydantic schema enforcement with constrained allowed values (not just valid JSON — semantically valid plans)
- Measuring **reliability metrics** as the primary contribution, not accuracy
- A **deterministic execution layer** that makes schema guarantees meaningful
- A **controlled baseline comparison**: same system, same datasets, free-form vs. schema

1.3. Scope Statement

Scope — Say This Explicitly in Your Introduction

“We intentionally scope to tabular classification to maintain experimental control. The contribution is the reliability framework, not the task itself. Tabular classification provides unambiguous ground truth, a well-defined pipeline space, deterministic sklearn execution, and isolates the variable we care about — schema enforcement — without confounding factors.”

Scope: Tabular classification only · UCI/OpenML datasets · scikit-learn models · Free models only (Groq + Google AI Studio free tiers)

1.4. Metrics That Matter for the Paper

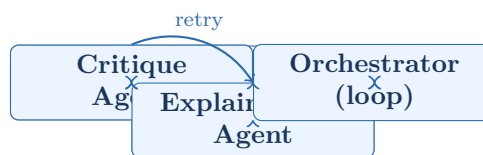
Metric	Definition	Measured where
Plan Validity Rate	% of runs where Analyzer produces a Pydantic-valid plan on first try	Experiment runner
Execution Success Rate	% of valid plans that execute without runtime error	Experiment runner
Score Variance	Std of final CV score across 5 runs on same dataset	Experiment runner
Iterations to Convergence	Mean iterations before Critique says “solved”	Experiment runner
Validator Issues Caught	# of logical errors caught by Plan Validator per run	RunArtifact log

2. System Architecture

2.1. Current Components (Working)

1. **Analyzer Agent** — LLM-driven, outputs AutoMLPlan Pydantic object. Uses Groq API (LLaMA 3.1) via OpenAI SDK with `response_format: json-object`.
2. **Implementation Agent** — Deterministic mapper, NOT an LLM. Executes AutoMLPlan using hardcoded sklearn/imblearn pipelines. Runs StratifiedKFold cross-validation.
3. **Critique Agent** — LLM evaluates results, produces suggestion. **Known weakness:** solved/not-solved decision is LLM-determined — must be fixed.
4. **Orchestrator** — Runs the loop: Analyzer → Implementation → Critique → repeat. Max 3 iterations. **Known weakness:** no memory between iterations.
5. **Gradio UI** — Basic interface, CSV upload + problem description.
6. **Schemas** — PreprocessingStep, ModelToTry, AutoMLPlan with validators.

2.2. Target Architecture (After Build)



2.3. Key Design Decisions — Do Not Change These

Architectural Rules

1. **Implementation Agent is NEVER an LLM.** Deterministic mapper only. This is what makes execution reproducible.
2. **Solved/not-solved is NEVER LLM-determined.** Threshold dict in `constants.py` only.
3. **Schema first, always.** Define all Pydantic models before writing any agent logic.
4. **No hardcoded model names or API keys** anywhere in agent files. Everything via `LLMConfig`.

3. Coding Practice — Follow This Order Every Time

For **every** new component you build, follow this exact sequence:

1. **Schema first** — Define all Pydantic models, Literals, validators before any logic
2. **Unit test the schema** — Test valid inputs pass, invalid inputs raise, aliases normalize
3. **Implement the component** — Code the agent/function against the already-validated schema
4. **Isolated test** — Test the component alone with mocked inputs, not the full pipeline
5. **Integration** — Wire into Orchestrator, run on one real dataset end-to-end
6. **Log inspection** — Manually read the RunArtifact JSON and verify it looks right
7. **Only then move on** — Never start the next component with a broken previous one

3.1. File Structure (Target)

```

automl_project/
  agents/
    base_agent.py           # BaseAgent abstract class
    analyzer_agent.py       # LLM-driven planner
    analyzer_baseline.py    # Free-form baseline variant
    critique_agent.py       # LLM critique + deterministic
  solved flag
    dataset_quality_agent.py
    plan_validator_agent.py
    feature_eng_agent.py
    hpo_agent.py
    explainability_agent.py
    implementation_agent.py # Deterministic, NO LLM
  schemas/
    automl_plan.py         # PreprocessingStep, ModelToTry,
  AutoMLPlan
  AgentCall
  quality_report.py        # DatasetQualityReport,
  ColumnQuality
    validation_result.py   # ValidationResult
    hpo_schema.py         # HPOCandidate, HPoSuggestion
    explainability.py      # FeatureImportance,
  ExplainabilityReport
  config/
    llm_config.py         # LLMConfig dataclass
    pipeline_config.py    # PipelineConfig (ablation flags
  )
    constants.py          # SOLVE_THRESHOLDS dict
  orchestrator.py
  data/
    load_datasets.py
    raw/                  # Downloaded OpenML datasets
  experiments/
    run_experiment.py
    collect_results.py
    analyze_results.py
  tests/
    test_llm_config.py
    test_run_artifact.py

```

```

    test_critique_determinism.py
    test_memory.py
    test_quality_agent.py
    test_plan_validator.py
    test_hpo_agent.py
    test_explainability.py
    test_baseline_parser.py
results/
    results_table.csv          # Final paper data
ui/
    app.py                    # Gradio interface

```

4. Sprint Plan — Week by Week

One Rule Above All

Week 3 is sacred. If you are behind on Week 2, cut scope (skip Feature Engineering or Explainability temporarily) but **never** delay the experiment runner. The results *are* the paper. Everything else is infrastructure.

4.1. Week 1 — Refactor Foundation

Goal: Clean architecture that everything else builds on. No new features.

Week 1 — Days 1–2: LLMConfig + BaseAgent

Schemas to define first:

- LLMConfig dataclass: provider: Literal["groq","google","ollama"], model: str, base_url: str, temperature: float, api_key_env_var: str (env var name, never the key itself)
- BaseAgent abstract class: __init__(self, config: LLMConfig, logs: list), abstract run(**kwargs)

Implement:

- Refactor AnalyzerAgent to extend BaseAgent — remove all hardcoded URLs and model names
- Refactor CritiqueAgent to extend BaseAgent — same

Tests to write (tests/test_llm_config.py):

- Instantiate configs for groq + google; verify OpenAI SDK client builds correctly from each
- Verify switching model is one line change
- Verify no API key appears in the config object (it reads from env)

Week 1 — Day 3: RunArtifact Schema

Schemas to define (all in schemas/run_artifact.py):

- AgentCall: agent_name: str, input_summary: str, output_summary: str, duration_seconds: float, timestamp: datetime
- IterationRecord: iteration_number: int, plan: AutoMLPlan, results:

```
dict, evaluation: str, solved: bool, agent_calls: list[AgentCall]
• RunArtifact: run_id: str (uuid4), dataset_path: str, problem: str,
  schema_version: str (hardcode "1.0"), llm_config: dict (serialized, no key),
  random_seed: int, iterations: list[IterationRecord], solved: bool,
  total_duration_seconds: float
```

Tests (tests/test_run_artifact.py):

- Construct a full fake RunArtifact manually
- Serialize to JSON, deserialize back
- Assert round-trip is byte-identical

Important: Do NOT wire into Orchestrator yet — just validate the schema is correct.

Week 1 — Day 4: Fix Critique Agent (Deterministic)**What to build in config/constants.py:**

- SOLVE_THRESHOLDS: dict keyed by metric, value is (small_threshold, large_threshold) where small = under 1000 rows
- Example: "f1_macro": (0.65, 0.72), "accuracy": (0.80, 0.85), "roc_auc": (0.75, 0.82)

What changes in CritiqueAgent:

- Add dataset_size: int to run() signature
- solved: bool computed in Python: score >= SOLVE_THRESHOLDS[metric][0 if size < 1000 else 1]
- LLM still generates critique_text: str and suggestion: str only
- Update CritiqueResult schema to NOT ask LLM for the boolean

Tests (tests/test_critique_determinism.py):

- Run same inputs 3 times — assert solved is identical every time
- Assert critique_text may vary (that is fine and expected)

Week 1 — Day 5: Plan Memory in Orchestrator**New schema (schemas/automl_plan.py or separate):**

- AttemptSummary: iteration: int, models_tried: list[str], best_score: float, metric: str, failure_reason: Optional[str]

Changes to AnalyzerAgent.run():

- Accept previous_attempts: list[AttemptSummary] = []
- When non-empty, inject structured block into prompt: “Do not suggest models from this list: {models_already_tried}”

Changes to Orchestrator:

- Build AttemptSummary after each iteration
- Pass growing list to next Analyzer call

Tests (tests/test_memory.py):

- Run 2 iterations on a fixed dataset
- Assert second plan contains no model from the first plan’s model list

End of Week 1 Checkpoint: Pipeline runs on Iris. RunArtifact JSON produced. Critique deterministic. No hardcoded model names anywhere. Git tag: v0.2-phase1-complete

4.2. Week 2 — New Agents

Goal: Dataset Quality, Plan Validator, HPO. These generate paper evidence — build them carefully.

Week 2 — Days 1–2: Dataset Quality Agent

Schemas (schemas/quality_report.py):

- ColumnQuality: name: str, missing_rate: float, dtype: str, n_unique: int, is_constant: bool
- DatasetQualityReport: n_rows: int, n_cols: int, columns: list[ColumnQuality], class_distribution: dict[str, float], imbalance_flag: bool (minority class < 20%), duplicate_row_rate: float, potential_leakage_columns: list[str], warnings: list[str]

Implement (agents/dataset_quality_agent.py):

- No LLM — pure pandas only
- Missing rate: `df[col].isna().mean()` per column
- Imbalance flag: check value counts of target, flag if minority < 20%
- Duplicates: `df.duplicated().mean()`
- Constant columns: `df[col].nunique() == 1`
- Leakage heuristic: correlation > 0.95 with target for numeric columns
- Populate warnings: list[str] as human-readable strings for Analyzer

Wire into Orchestrator:

- Runs before Analyzer
- DatasetQualityReport passed as parameter to `AnalyzerAgent.run()`
- Update Analyzer prompt: “The dataset quality report flagged: {warnings}. Ensure your plan addresses these.”

Tests (tests/test_quality_agent.py):

- Synthetic DataFrame with one constant column $\Rightarrow$ assert `is_constant=True`
- Synthetic DataFrame with 30% missing in one column $\Rightarrow$ assert `missing_rate  $\approx$  0.3`
- Imbalanced target (95/5 split) $\Rightarrow$ assert `imbalance_flag=True`

Week 2 — Days 3–4: Plan Validator Agent

Schema (schemas/validation_result.py):

- ValidationResult: passed: bool, issues: list[str], warnings: list[str]

Implement (agents/plan_validator_agent.py):

- No LLM — pure logic only
- Rule 1 (ordering): If `impute_missing` in steps, it must appear *before* `scale_features`. Violation $\rightarrow$ issue.
- Rule 2 (imbalance): If `imbalance_flag=True` and no `SMOTE/class_weight` step $\rightarrow$ issue.

- **Rule 3 (metric/imbalance):** If metric is accuracy and `imbalance_flag=True` → warning (misleading metric).
- **Rule 4 (dataset size):** If `n_rows < 500` and ensemble model selected → warning (overfitting risk).
- **Rule 5 (leakage):** If any column in `potential_leakage_columns` not in plan's drop list → issue.

Wire into Orchestrator:

- Runs after Analyzer, before Implementation
- If not `validation_result.passed`: feed issues back to Analyzer as correction prompt, retry once
- Log validator-triggered re-plans in `RunArtifact` — these are paper data points

Tests (`tests/test_plan_validator.py`):

- Construct a plan that violates each rule independently
- Assert the correct issue string appears in `ValidationResult.issues`
- Assert a correct plan passes with `passed=True` and empty issues

Week 2 — Day 5: HPO Agent

Schemas (`schemas/hpo_schema.py`):

- `HPOCandidate`: `model_name: str, hyperparameters: dict[str, Any], rationale: str`
- `HPOSuggestion`: `candidates: list[HPOCandidate]` — validator enforces `len == 3`

Implement (`agents/hpo_agent.py`):

- LLM-driven: prompt asks for exactly 3 hyperparameter configurations in JSON schema
- Constrained: maintain an allowlist dict of valid sklearn params per model (e.g., `"RandomForestClassifier": ["n_estimators", "max_depth", "min_samples_split", ...]`)
- Execute all 3 via Implementation Agent infrastructure, return best
- Scoped narrowly: no full grid search, exactly 3 structured candidates

Wire into Orchestrator:

- Activates only when `CritiqueResult.needs_tuning=True`
- Add `needs_tuning`: bool flag to `CritiqueResult` schema: True when score is within 5% of threshold but not solved

Tests (`tests/test_hpo_agent.py`):

- Mock LLM call, verify exactly 3 candidates always returned
- Verify invalid hyperparameter keys are rejected by schema
- Verify `HPOSuggestion` raises on fewer or more than 3 candidates

End of Week 2 Checkpoint: Run full pipeline on 3 datasets (Iris, Adult, Wine). Inspect `RunArtifact` JSON — verify all agents in `agent_calls`, quality warnings flow to Analyzer, validator fires on Adult (imbalanced). Git tag: `v0.3-agents-complete`

4.3. Week 3 — Experiments & Baseline

Goal: Collect the data the paper is built on. This is the most important week.

Week 3 — Day 1: Dataset Expansion

Download these 10 datasets via `openml.datasets.get_dataset(id)`:

Dataset	OpenML ID	Size	Balance	Type
Iris	61	Small	Balanced	Numeric, Multiclass
Wine	187	Small	Balanced	Numeric, Multiclass
Breast Cancer	15	Small	Imbalanced	Numeric, Binary
Pima Diabetes	37	Small	Imbalanced	Numeric, Binary
German Credit	31	Medium	Imbalanced	Mixed, Binary
Vehicle Silhouettes	54	Medium	Balanced	Numeric, Multiclass
Car Evaluation	40975	Medium	Multiclass	Categorical, Multiclass
Adult/Census	1590	Large	Imbalanced	Mixed, Binary
Bank Marketing	1461	Large	Imbalanced	Mixed, Binary
Jungle Chess	40691	Large	Balanced	Numeric, Binary

Write `data/load_datasets.py`:

- Returns `(df, target_col, problem_statement)` tuples
- Saves to `data/raw/`
- Write a standard `problem_statement` string per dataset — keep these **identical** across schema and baseline runs (controlled comparison)

Week 3 — Day 2: Build the Baseline

Implement `agents/analyzer_baseline.py`:

- Same interface as `AnalyzerAgent`, but prompt asks for free-form text
- Response parsed with a simple extractor: regex for model names, keyword matching for preprocessing
- If parsing fails → log `plan_valid: False`, return null plan
- Intentionally fragile — **the fragility is the point**

Define `BaselinePlan` (plain dataclass, not Pydantic):

- `raw_text: str, extracted_models: list[str], extracted_preprocessing: list[str], parse_success: bool`

Important: Baseline uses the **same Implementation Agent** — fair comparison; execution is identical when a plan parses successfully.

Tests (`tests/test_baseline_parser.py`):

- Give 10 sample free-form LLM responses (write these by hand)
- Assert parse success rate
- Count how many produce valid plans vs. failures — this number will appear in your paper

Week 3 — Days 3–4: Experiment Runner

Write `experiments/run_experiment.py`:

- Args: `dataset_id, problem_statement, n_runs: int, mode: Literal["schema", "baseline"]`

- Run full pipeline `n_runs=5` times
- Collect per run: `plan_valid: bool, execution_success: bool, final_score: float, n_iterations: int, total_duration: float, validator_issues_caught: int`
- Save each full RunArtifact to `experiments/artifacts/{dataset_id}/{mode}/run_{i}.json`

Write `experiments/collect_results.py`:

- Reads all saved artifacts
- Outputs `results/results_table.csv` with columns:
`dataset, mode, run, plan_valid, execution_success, score, n_iterations, validator_issues, duration`
- This CSV is directly your paper's results table

Run: 10 datasets × 5 runs × 2 modes = **100 total runs**. Run overnight.

Week 3 — Day 5: Results Analysis

Write `experiments/analyze_results.py`:

- Load `results_table.csv`
- Compute: validity rate per dataset (schema vs. baseline), execution success rate, mean±std score, mean iterations
- Print formatted table for copy-paste into paper
- Compute **variance ratio**: `baseline_std / schema_std` — if > 1.2 consistently, that is your reproducibility claim quantified

Eyeball the results honestly:

- Are there datasets where baseline does surprisingly well? Those go in the paper as limitations
- Is the story clean? A 20% improvement in validity rate across 10 datasets is publishable
- If results are weak, check: is the baseline parser too smart (making baseline look better than it should)?

End of Week 3 Checkpoint: `results_table.csv` exists with 100 rows. You can compute all 5 paper metrics. Git tag: `v0.4-experiments-complete`

4.4. Week 4 — Explainability, Ablation Flags & Poster

Week 4 — Day 1: Explainability Agent

Schemas (`schemas/explainability.py`):

- `FeatureImportance: feature_name: str, importance_score: float, rank: int`
- `ExplainabilityReport: method: Literal["feature_importance", "permutation"], top_features: list[FeatureImportance], decision_summary: str, model_name: str`

Implement (`agents/explainability_agent.py`):

- For tree-based models: use `.feature_importances_` directly
- For others: use permutation importance from sklearn (no extra dependency)
- `decision_summary` is LLM-generated: “Given these top features, the model appears

to decide based on...”

- Top 5 features maximum

Tests: Fit a simple `RandomForestClassifier` on Iris. Assert top 3 features returned. Assert scores sum to approximately 1.0 for tree models.

Week 4 — Day 2: Ablation Flags + Cleanup

Define `config/pipeline_config.py`:

- `PipelineConfig`: `use_quality_agent: bool, use_plan_validator: bool, use_hpo: bool, use_explainability: bool, use_memory: bool`
- Pass to Orchestrator; wrap each agent call in `if config.use_X`

Ablation study enabled by this:

- Full system vs. no-validator vs. no-memory vs. no-quality-agent
- Each ablation run uses the same experiment runner — just change the config

Cleanup:

- Bump `schema.version` to "1.1"
- Run final clean end-to-end test on all 10 datasets — fix any remaining bugs
- Ensure all `RunArtifact` JSONs produced by Week 3 are still readable with the new schema

Week 4 — Days 3–5: Poster

Poster structure (A0 portrait, 841 × 1189 mm):

1. **Title band (top):** Title, authors, institution, venue, QR code to arXiv
2. **Left column — Motivation:**
 - 2–3 sentences: why free-form LLM AutoML fails
 - One diagram: free-form response (partially parsed, error) vs. schema rejection (typed Pydantic error). Simple two-panel figure.
3. **Center column — Architecture:**
 - One clean diagram of your 6-agent pipeline with arrows
 - Schema objects shown as typed nodes between agents
 - Each agent box labeled with input/output schema name
4. **Right column top — Results:**
 - Your `results_table.csv` summary — the 4 key metrics, schema vs. baseline columns
 - Bold the cells where schema wins
5. **Right column bottom — Key Findings:**
 - 3–4 bullet points with concrete numbers
 - “Schema plans had X% higher validity rate. Baseline failed Y% of runs due to parsing errors. Score variance was Z× lower.”
6. **Bottom band — Conclusion + Future Work:** 3 sentences max

Poster tips:

- The architecture diagram is your most important visual — spend time on it

- Use consistent colors for schema objects (e.g., blue) vs. LLM calls (e.g., orange) throughout
- Font size minimum 24pt for body text, 36pt for headings (readable from 1.5m away)
- Use your own tool (PowerPoint, Inkscape, Canva, L^AT_EX beamerposter) — whatever you're fastest with

End of Week 4 Checkpoint: Poster exported as PDF. All agents implemented. Ablation flags working. Ready to write the paper.

4.5. Week 5 — Paper & Submission (Buffer)

Week 5 — Paper Writing

Paper structure (4 pages, AutoML-Conf Late-Breaking format):

1. **Introduction** (0.5 page): Problem, claim, contribution summary
2. **Related Work** (0.5 page): AIDE, AutoML-GPT, DS-Agent, MLAGentBench — explain what each misses that you measure
3. **System Design** (1 page): Architecture figure + schema design rationale. Explain why Implementation Agent is deterministic.
4. **Experiments** (1.5 pages): Datasets table, results table (your CSV), analysis of variance ratio
5. **Conclusion** (0.5 page): Answer the research question directly with your numbers

Submission targets:

- **AutoML-Conf 2026 Late-Breaking Track** — deadline **July 15, 2026**
- **arXiv** (cs.LG or cs.AI) — post simultaneously with submission
- **HAL** — deposit for institutional visibility (ESI Algiers)

5. LLM Provider Setup

5.1. Free Models to Use

Model	Provider	API base URL	Notes
LLaMA 3.3 70B	Groq (free tier)	<code>api.groq.com/openai/v1</code>	Already used
Gemini 2.0 Flash	Google AI Studio (free)	<code>generativelanguage.googleapis.com/v1beta/openai/</code>	OpenAI compatible

Run experiments on both. If schema-driven validity holds on both model families, the claim is stronger — it is not a quirk of one model. Your LLMConfig abstraction makes switching trivial.

6. Datasets Reference

Dataset	OpenML ID	Rows	Cols	Task	Coverage
Iris	61	150	4	Multiclass	Small, balanced, numeric
Wine	187	178	13	Multiclass	Small, balanced, numeric
Breast Cancer WI	15	569	30	Binary	Small, imbalanced, numeric
Pima Diabetes	37	768	8	Binary	Small, imbalanced, numeric
German Credit	31	1000	20	Binary	Medium, imbalanced, mixed
Vehicle Silhouettes	54	846	18	Multiclass	Medium, balanced, numeric
Car Evaluation	40975	1728	6	Multiclass	Medium, balanced, categorical
Adult/Census	1590	48842	14	Binary	Large, imbalanced, mixed
Bank Marketing	1461	45211	16	Binary	Large, imbalanced, mixed
Jungle Chess	40691	44819	6	Binary	Large, balanced, numeric

7. Reading List

7.1. Must Cite (Related Work)

- **AIDE** (Weco AI, 2024) — LLM agent for ML engineering, free-form code generation
- **MLAgentBench** (Huang et al., ICML 2024) — benchmarks LLM agents on ML tasks, measures accuracy only
- **AutoML-GPT** (arXiv 2023) — early LLM-based AutoML pipeline
- **DS-Agent** (arXiv 2024) — LLM agent for data science
- **Auto-sklearn 2.0** (Feurer et al., 2022) — position your work as orthogonal, not competing
- **FLAML** (Microsoft, 2021) — same as above
- **Outlines: Guided Text Generation** (Willard & Louf, 2023) — constrained decoding vs. your post-processing approach

7.2. Background Reading

- **AutoML: A Survey of the State-of-the-Art** (He et al., 2021, Knowledge-Based Systems)
- **AgentBench: Evaluating LLMs as Agents** (Liu et al., 2023) — agent evaluation methodology
- **OpenAI Structured Outputs blog post** (2024) — industry framing for your approach
- **Pydantic v2 docs** — model validators vs. field validators; ensure alias normalization is correct

8. Timeline Summary

Week	Main Output	Done When	Git Tag
1	Refactored codebase	Pipeline runs, RunArtifact produced, critique deterministic	v0.2
2	Quality + Validator + HPO	All agents pass isolated tests, wired into Orchestrator	v0.3
3	Experiment results	100 runs complete, <code>results_table.csv</code> exists	v0.4
4	Explainability + Poster	All agents working, poster exported as PDF	v0.5
5	Paper + submission	arXiv posted, AutoML-Conf submitted by July 15	v1.0

Submission Targets

- **AutoML-Conf 2026 Late-Breaking Track** — Deadline: **July 15, 2026** (Ljubljana, Slovenia)
- **AutoML-Conf 2026 ABCD Main Track** — Deadline: **April 30, 2026** (if results ready early)
- **arXiv** (cs.LG / cs.AI) — post simultaneously with any submission